

CO2 - ASSESSMENT TOOL 1

INTEGRATED EXPERIMENTS

NAME: ADHIL BIJUKUMAR

REG. NO:192411133

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4308

QUESTIONS:

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

Source Code:

```
<!DOCTYPE html>

<html>

<head>

<title>Responsive Layout</title>

<style>

*{

    margin:0;

    padding:0;

    box-sizing:border-box;

}

body{

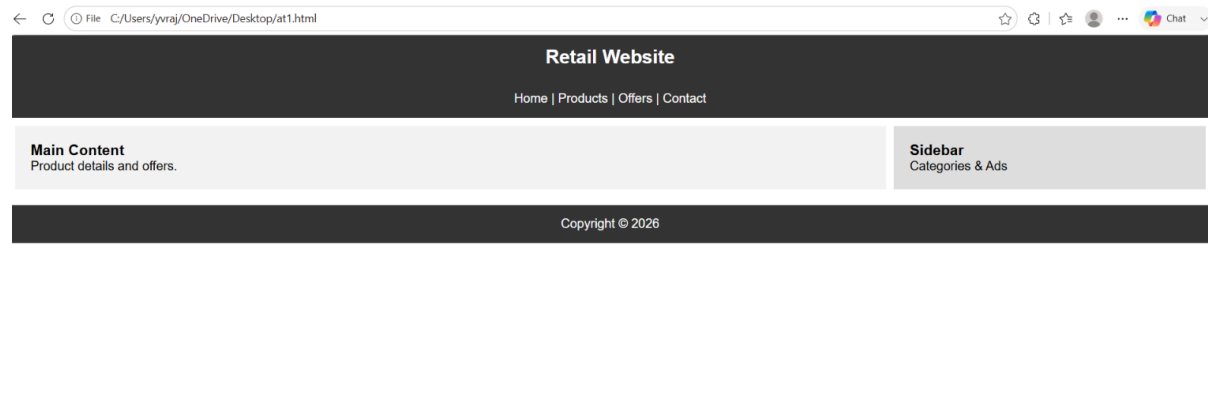
    font-family:Arial;

}
```

```
header,nav,footer{
    background:#333;
    color:white;
    text-align:center;
    padding:15px;

.container{
    display:flex;
    gap:10px;
    padding:10px;
}
.content{
    flex:3;
    background:#f2f2f2;
    padding:20px;
}
.sidebar{
    flex:1;
    background:#ddd;
    padding:20px;
}
footer{
    margin-top:10px;
}
@media(max-width:768px){
.container{
```


Output:



(a) Analyze the possible reasons for lack of responsiveness in the existing layout.

- Fixed-width layouts
- No media queries
- Large images without scaling
- Improper spacing and alignment
- No Flexbox or CSS Grid

(b) Design a responsive webpage layout using CSS Flexbox or Grid, ensuring proper alignment of header, navigation, content, and footer.

- Used CSS Flexbox for layout.
- Includes Header, Navigation, Content, Sidebar, and Footer.

(c) Demonstrate how the layout adapts to different screen sizes (mobile, tablet, desktop) using media queries.

- Desktop: Content and sidebar displayed side by side.
- Mobile/Tablet ($\leq 768\text{px}$): Content and sidebar stack vertically.

(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.

- Used padding, margin, gap, and flex properties.
- Elements are evenly aligned with proper spacing.

(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

- Responsive on mobile, tablet, and desktop.

- Better readability and user experience.
- Further improvements:
 - Use CSS Grid for more complex layouts.
 - Add responsive images.
 - Include a hamburger menu for mobile navigation.
 - Use Bootstrap or Tailwind CSS for faster responsive development.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

Source Code:

```
<!DOCTYPE html>

<html>

<head>

<title>Form Validation</title>

<script>

function validateForm() {

    let email = document.getElementById("email").value;

    let phone = document.getElementById("phone").value;

    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    let phonePattern = /^[0-9]{10}$/;

    document.getElementById("emailError").innerHTML = "";

    document.getElementById("phoneError").innerHTML = "";

    let valid = true;

    if (!emailPattern.test(email)) {

        document.getElementById("emailError").innerHTML = "Invalid Email";

        valid = false;
```

```

    }

    if (!phonePattern.test(phone)) {

        document.getElementById("phoneError").innerHTML = "Enter 10-digit Phone Number";

        valid = false;

    }

    return valid;

}

</script>

</head>

<body>

<h2>Registration Form</h2>

<form onsubmit="return validateForm()">

Email:

<input type="email" id="email" required>

<span id="emailError" style="color:red"></span>

<br><br>

Phone:

<input type="text" id="phone" required>

<span id="phoneError" style="color:red"></span>

<br><br>

<input type="submit" value="Register">

</form>

</body>

</html>

```

Output:

Registration Form

Email:

Phone:

(a) Analyze the issues in the existing form validation mechanism.

- Invalid email formats are accepted.
- Phone numbers may contain letters or incorrect lengths.
- No real-time validation.
- Incorrect data can be submitted.

(b) Design input fields for email and phone number with appropriate HTML attributes.

- Used type="email" for email.
- Used required attribute.
- Phone number input is validated using JavaScript.

(c) Implement JavaScript validation using Regular Expressions for both email and phone number.

- Email Regex:
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`
- Phone Regex:
`/^[0-9]{10}$/`

(d) Develop a mechanism to display real-time error messages for invalid inputs.

- Displays "Invalid Email" for incorrect email format.
- Displays "Enter 10-digit Phone Number" for invalid phone numbers.

(e) Evaluate the validation system and suggest enhancements for better user experience and security.

- Prevents invalid data submission.
- Improves data accuracy.
- Further enhancements:

- Validate while typing using the input event.
- Add password strength validation.
- Sanitize inputs on the server side.
- Use HTTPS and server-side validation for better security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

Source Code:

```
<!DOCTYPE html>

<html>

<head>

<title>CSS Layout Debugging</title>

<style>

*{

    margin:0;

    padding:0;

    box-sizing:border-box;

}

body{

    font-family:Arial;

}

.container{

    display:flex;

    flex-wrap:wrap;

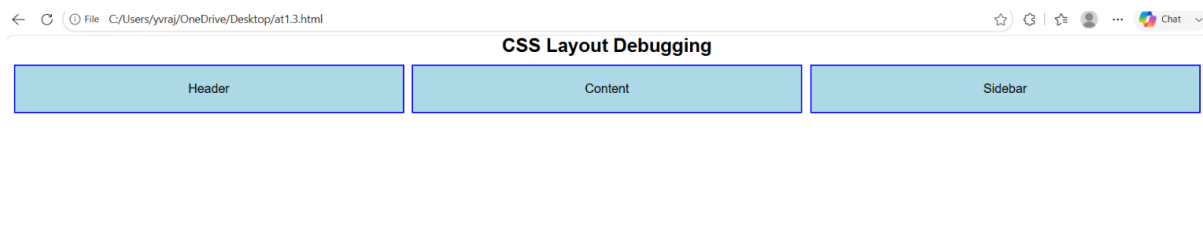
    gap:10px;

    padding:10px;
```



```
}  
  
.box{  
    flex:1;  
    min-width:200px;  
    background:lightblue;  
    padding:20px;  
    border:2px solid blue;  
    text-align:center;  
}  
  
@media(max-width:768px){  
    .container{  
        flex-direction:column;  
    }  
}  
  
</style>  
</head>  
<body>  
<h2 align="center">CSS Layout Debugging</h2>  
<div class="container">  
    <div class="box">Header</div>  
    <div class="box">Content</div>  
    <div class="box">Sidebar</div>  
</div>  
</body>  
</html>
```

Output:



(a) Analyze the CSS properties responsible for layout overlapping issues.

- Fixed width and height values.
- Incorrect use of position: absolute.
- Missing responsive design.
- No flex-wrap or media queries.
- Excessive margins or padding.

(b) Examine the role of the CSS box model (margin, border, padding, content) in layout design.

- Content: Main content area.
- Padding: Space inside the border.
- Border: Surrounds the padding and content.
- Margin: Space outside the border.

(c) Debug the layout using appropriate positioning techniques (relative, absolute, flex, grid).

- Used Flexbox for responsive layout.
- Added flex-wrap: wrap to avoid overlap.
- Used media queries for smaller screens.
- Avoided unnecessary absolute positioning.

(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.

- Added box-sizing: border-box.
- Used display: flex.
- Added gap for spacing.
- Used min-width and media queries for proper alignment.

(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

- Layout no longer overlaps when resized.
- Responsive on desktop, tablet, and mobile.
- Best practices:

- Use Flexbox or CSS Grid.
- Avoid fixed widths where possible.
- Use responsive units (% , rem, vw).
- Test layouts on different screen sizes.
- Keep consistent spacing with margin, padding, and gap.